

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

MongoDB Python API

Készítette: Hauer Attila Árpád

Neptunkód: JJL4WE

Dátum: 2026.05.13

Tartalom

1 MongoDB Pymongo API	3
1.1 MongoCilent	3
1.2 Adatbázis és Kollekciónk	3
1.4 Alapvető műveletek	3
1.5 Aggregáció	4
2. MongoDB megvalósítása Python API-val	4
2.1 Adatmodell kialakítása	4
2.2 Adatok inicializálása	4
2.3 Adatok lekérdezése	5
2.4 Update	5
2.5 Delete	5
2.6 Kollekciónk összekapcsolása	6
2.7 Aggregációs műveletek	6
3. Saját XDM modell és XML Schema-ra való konvertálás	7
3.1 Modell bemutatása	7
3.2 Konvertálás	7
3.3 komplex típusok	8
3.4 egyszerű típusok	8
3.5 többértékű tulajdonságok	9
3.6 Összetett adatszerkezetek	9
3.7 Kulcsok definiálása	9
3.8 Idegen kulcsok	9
3.9 1:1 kapcsolatok kulcsa	10

1 MongoDB Pymongo API

A MongoDB Python API-ja a Pymongo könyvtáron keresztül teszi lehetővé az adatbázis kezelését. Ez a driver biztosítja a kapcsolatot, és az adatkezeléshez szükséges metódusokat.

1.1 MongoClient

A szerverhez való kapcsolódásért a MongoClient objektum felel.

```
client = mongo.MongoClient("mongodb://localhost:27017/")
```

Ez a komponens kezeli a kapcsolatot az adatbázis szerverrel.

1.2 Adatbázis és Kollekciónk

Az adatbázisnak és a kollekcióknak a kiválasztása ugyanolyan szintaktikával történik. A kollekciók reprezentálják a dokumentum tároló egységeket. Hasonló szerepet töltenek be mint a tábla a relációs adatbázisban, de mivel a MongoDB schema less ezért a struktúrájuknak nem feltétlen kell megegyezniük, így rugalmasabb az adatkezelés.:

```
db = client["VendeglatasDB"]  
  
#Collection létrehozása  
etterem_coll = db["etterem"]  
foszakacs_coll = db["foszakacs"]
```

1.4 Alapvető műveletek

A pymongo támogatja az alapvető CRUD műveleteket

Create: insert_one(), insert_many()

Read: find(), find_one()

Update: update_one(), update_many()

Delete: delete_one(), delete_many()

1.5 Aggregáció

A MongoDB-ben aggregációs műveletekhez pipeline-t kell használni, mely segítségével megvalósíthatóak a sum(), count(), min(), max(), avg() műveletek.

2. MongoDB megvalósítása Python API-val

2.1 Adatmodell kialakítása

A feladat során egy vendéglátóipari rendszert modelleztem. A dokumentumalapú megközelítés lehetővé tette a hierarchikus adatok tárolását:

- **Étterem:** Tartalmazza a nevet, csillagok számát és egy beágyazott cím objektumot (város, utca, házszám).
- **Főszakács:** Tartalmazza a személyes adatokat (név, életkor), az iskolai végzettségeket listaként ([]), valamint egy hivatkozást az étteremre (e_f mező).

2.2 Adatok inicializálása

A kollekciós típusok létrehozása után egy Create műveletet hajtottam végre mellyel feltöltöttem az üres kollekciókat a feladathoz szükséges adatokkal. A tömeges adatbetöltés az insert_many() függvénnyel valósítottam meg, amely Python listákat vár, bennük szótár objektumokkal.

```
ettermek_adatok = [  
    {  
        "id": "e1",  
        "nev": "Aranyhal Étterem",  
        "cim": {  
            "varos": "Miskolc",  
            "utca": "Széchenyi u.",  
            "hazszam": 107  
        },  
        "csillag": 3  
    }...]
```

2.3 Adatok lekérdezése

Az adatok lekérdezése a Pythonban `find()`-al vagy `find_one()`-al lehetséges. A szűréseket python szótárak segítségével definiáltam.

Teljes lista lekérdezése:

```
print("\n--- 2.a) Összes főszakács ---")
for fozsakacs in fozsakacs_coll.find():
    print(fozsakacs)
```

Szűrés konkrét ID-re:

```
e2_etterem = etterem_coll.find_one({"id": "e2"})
print(e2_etterem)
```

Összehasonlítás: Az 5 csillagnál kevesebb értékelésű éttermek

```
for etterem_csillag in etterem_coll.find({"csillag": {"$lte": 4}}):
    print(etterem_csillag)
```

2.4 Update

Az update-el értéket lehet módosítani egy kollekcióban. Update során meg kell adni először hogy mit akarunk módosítani majd a „\$set” paraméterrel lehet meghatározni hogy az adott kollekció melyik mezőjét mire módosítsuk.

```
etterem_coll.update_one(
    {"_id": "e1"},
    {"$set": {"csillag": 4}}
)
```

2.5 Delete

Megvalósítottam egyedi törlést (`_id` alapján) és feltételes törlést is, ahol a 30 évnél fiatalabb főszakácsokat távolítottam el a rendszerből a `$lt` (less than) operátor

használatával. A törlés során nem adja vissza hogy melyik objektumot töröltük csak azt, hogy hányat ezt a `.deleted_count`-tal lehet lekérdezni.

```
torles_eredmeny = foszakacs_coll.delete_many(
    {"eletkor": {"$lt": 30}}
)
print(f"Törölt főszakácsok száma: {torles_eredmeny.deleted_count}")
```

2.6 Kollektiók összekapcsolása

Bár a MongoDB egy NoSQL adatbázis a lookup segítségével megvalósítható a JOIN-hoz hasonló kapcsolat.

```
pipeline = [
    {
        "$match": {
            "vegzettseg": "Szakközépiskola"
        }
    },
    {
        "$lookup": {
            "from": "etterem",
            "localField": "e_f",
            "foreignField": "_id",
            "as": "etterem_adatok"
        }
    }
]
```

2.7 Aggregációs műveletek

Statisztikai számításokat az `aggregate()` metódussal lehet végezni. Példaként a főszakácsok átlagos életkorának meghatározását valósítottam meg a `$group` és `$avg` operátorok segítségével:

```
pipeline_avg = [
    {
        "$group": {
            "_id": None,
            "atlagEletkor": {"$avg": "$eletkor"}
        }
    }
]
```

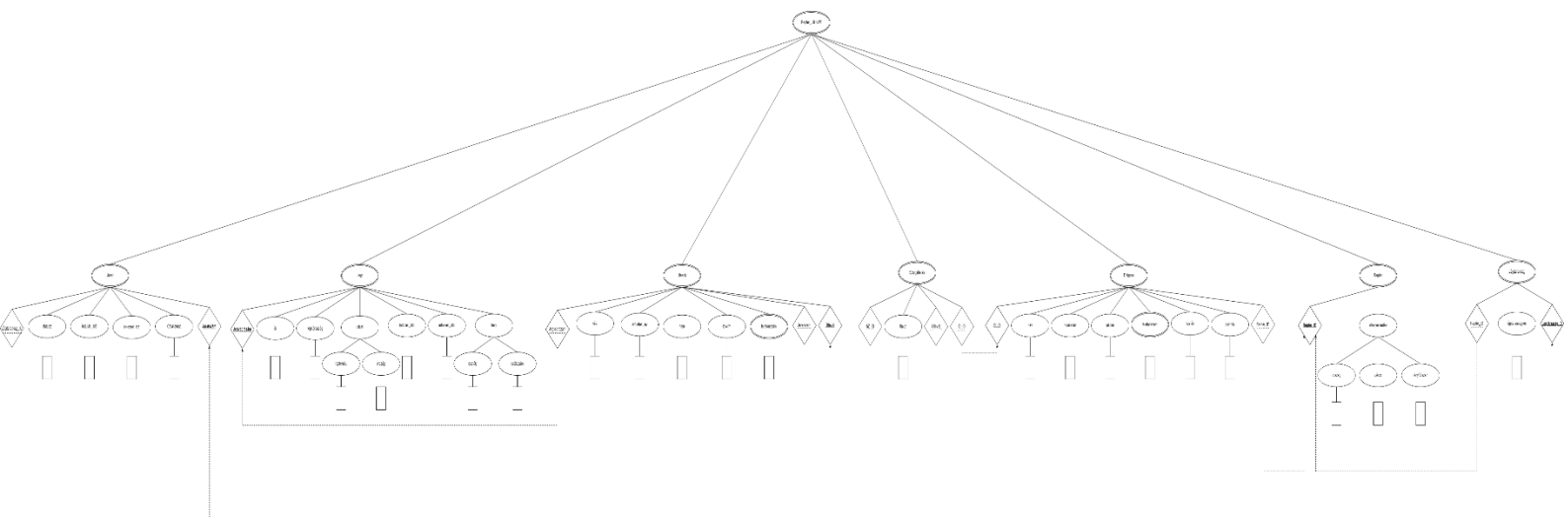
```
]
atlag_eredmeny = list(foszakacs_coll.aggregate(pipeline_avg))
```

3. Saját XDM modell és XML Schema-ra való konvertálás

Afeladat célja az volt, hogy az XML adatmodell alapját képező XDM modellből egy formális, validációra alkalmas XML Schema hozzak létre.

3.1 Modell bemutatása

Maga az XML Schema az alapjául szolgáló XDM dokumentum alapján készült. Az XDM modellem egy reptér fő entitásait, illetve azok kapcsolatait ábrázolja.



3.2 Konvertálás

Az XML-nek csak egyetlen gyökéreleme lehetséges, így én a modellben is látható REPTÉR-t választottam gyökérelemnek. Az összes többi ebben helyezkedik el.

```
<xs:element name="Reptér_JJL4WE">
```

3.3 komplex típusok

Az XDMmodell minden fő entitását külön complexType típusként valósítottam meg.

```
<xs:complexType name="járatTípus">
  <xs:sequence>
    <xs:element ref="célállomás"/>
    <xs:element ref="indulási_idő"/>
    <xs:element ref="érkezési_idő"/>
    <xs:element ref="státusz"/>
  </xs:sequence>
  <xs:attribute name="Járatszám" type="xs:int"/>
  <xs:attribute name="Légitársaság_ID" type="xs:int"/>
</xs:complexType>
```

Például a járat típus melyhez tartozik a célállomás, indulási idő, érkezési idő, státusz meg az elsődleges illetve az idegen kulcs.

Ezek az adattagok külön kerültek definiálásra mely során minden mezőhöz egy megfelelő adattípust választottam. Ez biztosította az XML dokumentum helyes validációját.

```
<xs:element name="célállomás" type="xs:string" />
<xs:element name="érkezési_idő" type="idoTípus" />
<xs:element name="indulási_idő" type="idoTípus" />
<xs:element name="státusz" type="xs:string" />
```

3.4 egyszerű típusok

Saját típusokat Simple type-ként hoztam létre, ezzel korlátoztam a beleírandó adatok lehetőségét vagy formátumát. Példa erre a nem az utasoknál, ahol vagy férfi, vagy nő az elfogadott érték.

```
<xs:simpleType name="nemTípus">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Férfi"></xs:enumeration>
    <xs:enumeration value="Nő"></xs:enumeration>
  </xs:restriction>
</xs:simpleType>
```


3.5 többértékű tulajdonságok

Többértékű előfordulás esetén a maxOccurs-el jeleztem.

```
<xs:element name="telefonszám" type="telefonTípus" minOccurs="0"
maxOccurs="unbounded"/>
```

3.6 Összetett adatszerkezetek

Az összetett adatszerkezeteket a feladat során komplex type-al valósítottam meg.

```
<xs:complexType name="reptérTípus">
  <xs:sequence>
    <xs:element name="elhelyezkedés">
      <xs:complexType>
        <xs:sequence>
          <xs:element ref="ország"/>
          <xs:element ref="város"/>
          <xs:element ref="irányítószám"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
  <xs:attribute name="Reptér_ID" type="xs:int"/>
</xs:complexType>
```

3.7 Kulcsok definiálása

Minden entitásnak létrehoztam egy elsődleges kulcsot. Ezt az xs:key-el valósítottam meg. Pl:

```
<xs:key name="Járatszám_kulcs">
  <xs:selector xpath="Járat"/></xs:selector>
  <xs:field xpath="@Járatszám"/></xs:field>
</xs:key>
<xs:key name="Jegy_kulcs">
  <xs:selector xpath="Jegy"/></xs:selector>
  <xs:field xpath="@Jegysorszám"/></xs:field>
</xs:key>
```

3.8 Idegen kulcsok

Az idegen kulcsokat az xs:keyref-el valósítottam meg.

```
<xs:keyref name="Járat-Utasok_kulcs" refer="Járatszám_kulcs">
  <xs:selector xpath="Utasok"/></xs:selector>
  <xs:field xpath="@Járatszám"/></xs:field>
```

```
</xs:keyref>  
<xs:keyref name="Jegy-Utasok_kulcs" refer="Jegy_kulcs">  
  <xs:selector xpath="Utasok"></xs:selector>  
  <xs:field xpath="@Jegysorszám"></xs:field>  
</xs:keyref>
```

3.9 1:1 kapcsolatok kulcsa

Az egy-egy kapcsolatok esetén a kulcsot xs:unique-al hoztam létre.

```
<xs:unique name="Jegy-Utasok_egy_egy">  
  <xs:selector xpath="Utasok"></xs:selector>  
  <xs:field xpath="@Jegysorszám"></xs:field>  
</xs:unique>
```